TrainBooking API — Request → Response Flow

How a request travels through the layers, using `GET /api/bookings` as the example.

Last updated 2026-11-04 · .NET 8 · FastEndpoints · MediatR · EF Core (Pomelo/MySQL) · Redis · OpenTelemetry/Prometheus

> **Scope of this document.** Everything marked LIVE exists in the code today. The Redis step marked HYPOTHETICAL is **not implemented yet**: Redis is registered (`IDistributedCache`) but no code reads or writes the cache. It is described here as the intended design.

1. Layers and dependency direction

The solution follows Clean Architecture / DDD. Dependencies always point inward: **Domain** depends on nothing, **Application** depends on Domain, **Infrastructure** implements interfaces declared in Domain/Application, and **Api** is the composition root that wires everything together at startup.

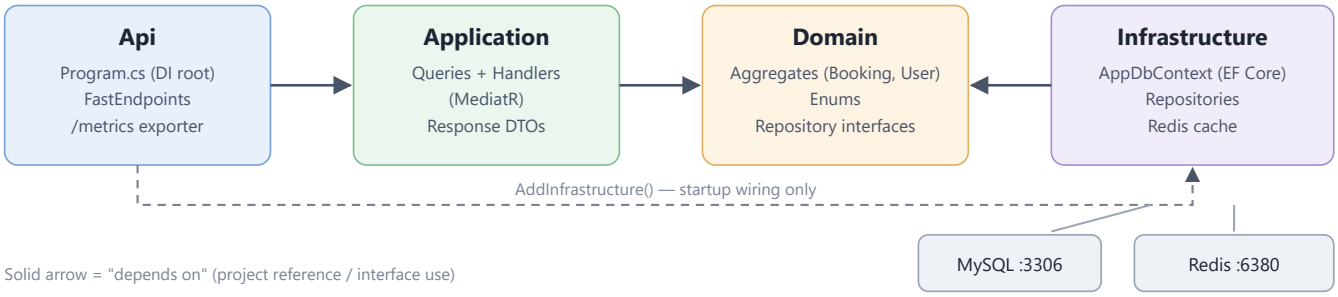


Figure 1 — Project dependencies. Domain is the center; Infrastructure depends on Domain, never the reverse.

2. Sequence: GET /api/bookings with a Redis cache

The cache follows the **cache-aside** pattern: the handler asks Redis first; on a hit it returns immediately, on a miss it loads from MySQL through the repository and stores the result in Redis for the next request.

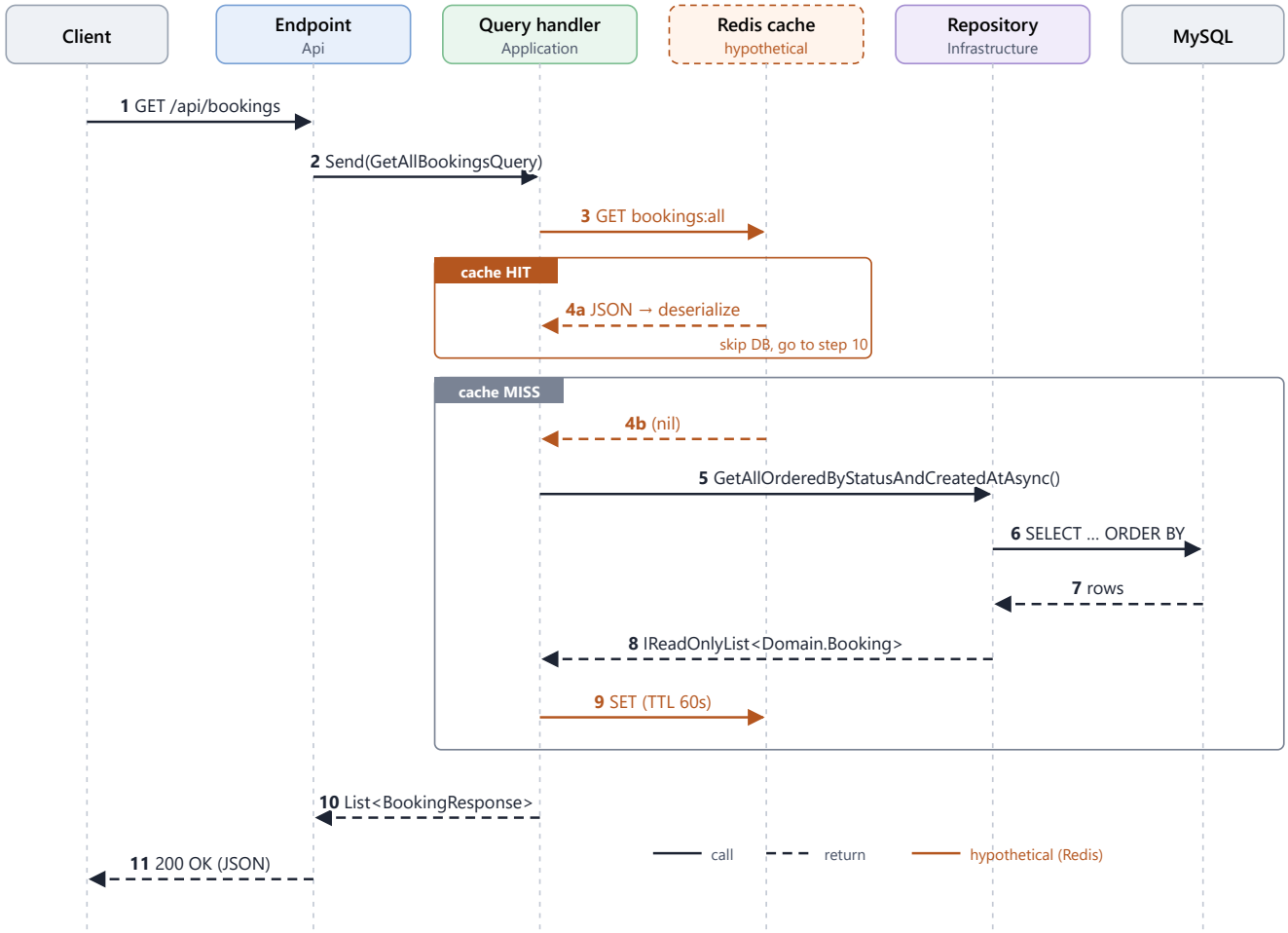


Figure 2 — Sequence diagram. Orange steps (3, 4a, 4b, 9) are the planned Redis cache; all other steps run today.

3. Step by step

1. **Kestrel receives the request** LIVE
   The HTTP request arrives on port 7777 and enters the ASP.NET Core middleware pipeline. A new **DI scope** is created for this request; every scoped service below (DbContext, repositories, handlers) lives only for this request. OpenTelemetry instrumentation starts timing the request.
   TrainBooking.Api/Program.cs

2. **Routing → FastEndpoints endpoint** LIVE
   The route `/api/bookings` matches `GetAllBookingsEndpoint`. FastEndpoints constructs it and injects MediatR's `ISender`. `AllowAnonymous()` means no authentication step; the endpoint has no request body, so there is no binding/validation step. `HandleAsync` sends a `GetAllBookingsQuery`. The endpoint knows nothing about databases or caches.
   TrainBooking.Api/Endpoints/GetAllBookingsEndpoint.cs

3. **MediatR dispatches to the query handler — and checks the cache** LIVE CACHE: HYPOTHETICAL
   MediatR resolves `GetAllBookingsQueryHandler` (registered by `AddApplication()`). In the cached design, the handler first asks the cache for key `TrainBooking:bookings:all` (the `TrainBooking:` prefix comes from `InstanceName` in the Redis registration).
   TrainBooking.Application/Bookings/Queries/GetAllBookings/GetAllBookingsQueryHandler.cs

4. **Cache hit or miss** HYPOTHETICAL
   Hit (4a): Redis returns the cached JSON; the handler deserializes it into `List<BookingResponse>` and jumps straight to step 10. MySQL is not touched.
   Miss (4b): Redis returns nothing; the handler continues to the repository.

5. **Repository call** LIVE
   The handler calls `IBookingRepository.GetAllOrderedByStatusAndCreatedAtAsync(ct)`. The interface is declared in **Domain**; DI supplies the `internal sealed BookingRepository` from Infrastructure plus a scoped `AppDbContext`.
   TrainBooking.Domain/Bookings/IBookingRepository.cs · TrainBooking.Infrastructure/Repositories/BookingRepository/BookingRepository.cs

6. **EF Core builds SQL and queries MySQL** LIVE
   The LINQ query `Bookings.AsNoTracking().OrderBy(Status).ThenBy(CreatedAt)` is translated by EF Core + Pomelo, targeting the MySQL version from configuration (`Database:ServerVersion` = `8.0.46-mysql`), so no extra round-trip is spent detecting the version. MySqlConnector takes a connection from the pool (connection string from user-secrets) and runs:

   ```
   SELECT b.Id, b.BookingCode, b.CreatedAt, b.ExpiresAt, b.Status, b.TotalAmount, b.UserId
   FROM Bookings AS b
   ORDER BY b.Status, b.CreatedAt
   ```

7. **Rows become persistence entities** LIVE
   EF materializes rows into the scaffolded `Infrastructure.Entity.Booking`. `AsNoTracking` skips change tracking because this is a read-only query.

8. **Persistence entity → domain aggregate** LIVE
   The repository maps each row with `Domain.Booking.Rehydrate(...)`, converting the `Status` string into the `BookingStatus` enum. An unknown status throws `InvalidOperationException`.
   TrainBooking.Domain/Bookings/Booking.cs

9. **Store the result in the cache** HYPOTHETICAL
   The handler maps the aggregates to `BookingResponse`, serializes them to JSON and writes them to Redis with a short TTL (e.g. 60 seconds) so the next request becomes a cache hit.

10. **Domain → DTO** LIVE
    The handler returns `IReadOnlyList<BookingResponse>` (enum turned back into a string), so domain objects never leak out of the API.

11. **Response** LIVE
    `SendOkAsync` serializes the list to camelCase JSON and returns **HTTP 200**. The DI scope is disposed: `AppDbContext` is released and the MySQL connection returns to the pool. OpenTelemetry records `http_server_request_duration_seconds{http_route="/api/bookings"}`.

4. How data changes shape between layers

Layer	Type	Notes
MySQL	rows in table `Bookings`	Status stored as a string
Infrastructure	`Infrastructure.Entity.Booking`	Scaffolded persistence model, public setters
Domain	`Domain.Bookings.Booking`	Aggregate root, private setters, `BookingStatus` enum
Application	`BookingResponse` (record)	DTO returned by the handler; also the value cached in Redis
Redis HYPO	JSON string	Key `TrainBooking:bookings:all`, TTL ~60 s
Api / HTTP	JSON body	camelCase, HTTP 200

5. Planned Redis design HYPOTHETICAL

Where the code would live

- **Application**: an `ICacheService` abstraction (`GetAsync<T>`, `SetAsync<T>`, `RemoveAsync`). Handlers depend on this interface, not on Redis.
- **Infrastructure**: `RedisCacheService` implementing it on top of the already-registered `IDistributedCache` (JSON serialization, TTL options).
- **Alternative**: a MediatR pipeline behavior that caches any query implementing an `ICachedQuery` marker, keeping handlers free of cache code.

Rules to keep it correct

- **Cache DTOs, not entities** — cache `BookingResponse`, never EF entities or aggregates.
- **Invalidate on write** — any command that creates, confirms or cancels a booking must remove `bookings:all`; the TTL is only a safety net.
- **Redis is optional** — if Redis is down (`abortConnect=false`), treat errors as a cache miss and fall back to MySQL instead of failing the request.
- **Do not cache seat availability** for booking decisions; double-booking must be prevented by the database (unique constraint), not by a cache.

6. Observability path

Every request is measured by OpenTelemetry and exposed at `/metrics`. Prometheus (`train_prometheus`, port 9090) scrapes it every 15 seconds via `host.docker.internal:7777`, and Grafana (port 3000) shows it on the provisioned **TrainBooking API** dashboard (request rate, p95 latency, status codes, .NET runtime).

7. Known gaps in the current flow

Item	Effect today
No global exception handler	A DB error or unknown status returns an empty HTTP 500.
Polly `BookingPipeline`	Registered but not used anywhere in the flow.
Redis	Registered and reachable, but not used yet (this document's orange steps).
No pagination	`GET /api/bookings` returns every row.